Automated Smart Parking System with Computer Vision and Crowdsourcing

Sonali Bhoite
Department of Information Technology
Vishwakarma Institute of Technology,
Pune, India
sonali.bhoite@vit.edu

Riddhi Mirajkar
Department of Information Technology
Vishwakarma Institute of Technology,
Pune, India
riddhi.mirajkar@vit.edu

Tejas Wanole
Department of Information Technology
Vishwakarma Institute of Information Technology,
Pune, India
tejas.22310001@viit.ac.in

Rohan Nemade
Department of Information Technology
Vishwakarma Institute of Information Technology,
Pune, India
rohan.22310412@viit.ac.in

Nirant Kale
Department of Information Technology
Vishwakarma Institute of Information Technology,
Pune, India
nirant.22310432@viit.ac.in

Durvesh Chavan
Department of Information Technology
Vishwakarma Institute of Information Technology,
Pune, India
durvesh.22310665@viit.ac.in

***Abstract*—Urban parking inefficiency fuels widespread congestion, wasted time, and environmental strain as drivers navigate crowded streets seeking free spaces. This paper details the architecture and functionality of an implemented Automated Smart Parking System (ASPS) that harmonizes Computer Vision (CV) for real-time slot detection with gamified crowdsourcing to democratize parking intelligence. The framework provides live availability maps via a responsive web application, enabling filtered searches and inviting users to contribute data on public or private zones through incentivized participation. By fusing precise technology with collective civic input, the system reduces search time, curbs emissions, alleviates traffic burdens, and fosters community-driven urban stewardship transforming parking from a daily frustration into a collaboratively optimized resource.**

***Keywords*—ASPS, CV, Smart Parking System (SPS), Crowdsourcing, Real-time Parking Detection, Web Application, Gamified Incentives, Hybrid Sensing, Traffic Decongestion, Emission Reduction, MERN Stack**

I. Introduction

The relentless pace of global urbanization has transformed cities into vibrant hubs of commerce and culture, but it has also given rise to a set of complex infrastructural challenges. Among the most pervasive and frustrating of these is the problem of inefficient urban parking. The daily ritual of circling city blocks in search of an available parking space is more than a mere inconvenience. It is a significant contributor to traffic congestion, economic waste, and environmental degradation. Studies indicate that drivers can spend an average of 20-40 minutes per trip searching for parking, a seemingly minor delay that, when aggregated across millions of vehicles, generates an enormous collective impact. This search traffic is estimated to constitute up to 30% of congestion in urban centers, leading to gridlock, increased driver stress, and lost productivity.

The consequences extend far beyond lost time. Environmentally, this cruising for parking results in an estimated 1.3 million tons of excess CO_2 emissions annually in major metropolitan areas alone, reduced efforts to create sustainable and healthy cities. Economically, the wastage of fuel represents a significant financial drain on consumers and contributes to a nation's energy inefficiency. Furthermore, the unreliability of parking availability acts as a liability to commerce, discouraging potential customers from visiting downtown business districts. Traditional parking management systems often reliant on manual patrols, static signage, and outdated metering technology are ill-equipped to handle the dynamic nature of modern urban mobility. They lack the capacity to provide real-time information, leading to a fundamental disconnect between parking supply and demand.

In response to these challenges, the concept of the "Smart City" has emerged, envisioning an urban landscape where technology is seamlessly integrated to optimize resources, enhance services, and improve the quality of life for citizens. A critical component of this vision is the development of SPS. This paper details the architecture, implementation, and projected impact of such a solution: the **ASPS**. The system is not a theoretical proposal but a fully implemented software solution designed to tackle the parking problem through a novel, hybrid approach that integrates high-precision technology with

community-driven data collection.

ASPS is built upon the synergistic combination of two powerful methodologies: autonomous CV and gamified crowdsourcing. For structured environments like parkings and unstructured streets, we deploy a standalone Python-based CV service that utilizes the YOLOv8 object detection model. This service provides highly accurate, real-time monitoring of individual parkings, detecting vehicle presence and communicating availability changes instantly via WebSockets. This ensures a reliable stream of data where infrastructure can be readily installed.

However, the true complexity of urban parking lies in the vast, unstructured network of on-street spaces, public areas, and informal lots where fixed sensors or cameras are impractical. To address this, ASPS incorporates a robust crowdsourcing mechanism within its user-facing web application. We empower citizens to become active participants in mapping their city's parking landscape. Through a simple and intuitive interface, users can report the parking lot availability and identify no-parking zones. To encourage sustained engagement, we have integrated a gamified rewards system where users earn in-app coins for their contributions, which can be redeemed within the ecosystem. This approach not only dramatically expands the system's coverage but also fosters a sense of community ownership and civic responsibility, playing a vital role in developing Smart Cities.

This entire ecosystem is powered by a modern, scalable technology stack, React and TypeScript frontend for a responsive user experience, and Node.js backend with MongoDB for robust data management. The system provides a unified, dual-role web application that serves both the general public (drivers/contributors) and system administrators, ensuring a seamless experience from finding a parking spot to managing the city-wide network.

This paper presents a comprehensive overview of the ASPS. Section II reviews the existing literature on smart parking solutions, identifying the gaps our system aims to fill. Section III provides a detailed description of the system architecture and implementation, covering the frontend, backend, and CV components. Section IV discusses the system's performance based on simulated testing and outlines the projected results in terms of reduced search times and user engagement. Finally, Section V concludes with a summary of our contributions and outlines future directions for development. Through this work, we demonstrate a practical, cost-effective, and scalable solution that transforms the daily frustration of parking into a collaborative, data-driven, and efficient urban service.

II. Literature Review

The field of SPS addresses the growing challenges of urban congestion, parking scarcity, and environmental impact. Giri introduces a "Smart Parking Assistant" that integrates a YOLOv5l model for real-time parking slot detection with a conversational chatbot interface using Rasa NLP. This system offers an intuitive interaction method but remains limited to structured parking lots, omitting crowdsourced data and unregulated public or illegal parking spots. This highlights an early effort toward combining CV and user interaction in SPS, though with notable constraints on scalability, user interactivity, visual representation and adaptability. [1]

Building on optimization strategies, Vantagudi et al. present an efficient parking management system aimed at optimizing space usage and reducing emissions. Their approach leverages priority-based algorithms and IoT-based ventilation sensors to manage cloud-based dynamic allocation of parking spaces. However, their methodology relies heavily on hardware infrastructure, focuses on air-quality and lacks real-time availability check or consideration of driver behavior, which are essential for adaptive decision-making in dynamic urban environments. [2]

Ahad and Kidwai's study offers a complementary perspective by focusing on driver preferences toward parking guidance and information systems. Their survey-based research reveals that most drivers prefer mobile or web applications for parking information over static signage. This aligns with the broader trend toward user-centric SPS design, and influences the development of systems that emphasize real-time information, dynamic filtering, and interactive interfaces for improved user experience. [3]

The sensing component of SPS has evolved significantly. Al-Obeidallah and Katheeri propose an IoT-based mobile application that uses ultrasonic and IR sensors to detect parking spaces. Their solution enables real-time map updates and payment integration through a mobile interface. However, such approaches tend to incur high infrastructure costs and are unable to capture illegal or informal parking spots, revealing a critical limitation of relying solely on fixed IoT hardware and introduces scalability issues. [4]

Unlike traditional sensor-based systems, Cura introduces a dual-module solution that integrates SMS queries with map-based interfaces to simplify the parking experience. The system leverages GSM modules alongside mapping APIs to enable spot reservation and availability checks via SMS. This hybrid approach eliminates the need for expensive IoT hardware by relying on widely accessible mobile communication tools, allowing users to interact with the system using text messages while receiving map-based directions in return. Such a design offers a cost-effective, scalable alternative to infrastructure-heavy systems and stands out for its use of completely distinct modules for communication and navigation. [5]

Zheng et al. focus on a hybrid crowdsensing model that leverages vehicular cameras combined with a Dynamic Gap Reduction Algorithm for roadside parking detection. Their approach demonstrates the potential for on-street space coverage without heavy infrastructure investment. However, it requires vehicle-mounted hardware, and lacks integration with administrative parking systems, which limits centralized oversight and broader applicability. [6]

Privacy and security challenges are central to modern SPS design. Yu et al. propose a "Privacy-Preserving Dynamic Multidestination Smart Parking" system based on blockchain technology and zero-knowledge proofs. Their encrypted spot

matching solution aims to ensure transparency and data security. While this approach offers a strong theoretical framework, its complexity and absence of real-time availability tracking highlight the trade-off between security and practical usability in SPS which lacks scalability. [7]

An important contribution to practical and functional SPS is provided by Higuchi and Oguchi, who study vision-based vehicular crowdsensing for live parking availability monitoring. Their solution uses federated learning to process in-vehicle camera data and updates real-time maps. Though innovative, this method raises privacy concerns and does not address core functionalities like reservations and payments, which are critical for widespread adoption. [8]

Similarly, Patel et al. present a web-based allocation system that focuses on centralized public parking management with a reservation system and dynamic pricing mechanism. Despite its functional simplicity, the system's lack of coverage for illegal parking again point to gaps in flexibility and real-time data handling that are needed in dynamic urban environments. Moreover, real-time accuracy depends on sensor integration. [9]

Finally, Ruzicka et al. provide a case study of the Liberec housing estate, which underscores the particular difficulties of parking in dense residential zones. Their spatial analysis and "parking-sharing" platform focus on local solutions without real-time tracking. This case highlights the necessity for hybrid models that combine fixed sensors, crowdsensing, and adaptive software to address diverse urban challenges effectively. [10]

III. System Architecture and Implementation

A. Overview of the Solution

The **ASPS** is an integrated, modular, and scalable framework built on the MERN stack (MongoDB, Express.js, React, Node.js). Using computer vision (CV) and gamified crowdsourcing, it is used to manage parking areas in both structured (lots, garages) and unstructured (streets, public areas) environments.

B. System Architecture

The implemented system comprises three primary layers: a frontend client, a backend server, and a computer vision service.

1) Frontend Client Layer: The primary point of interaction for all users is a responsive single-page application (SPA) built with **React** and **TypeScript**. This frontend client layer is designed to be intuitive and feature-rich. Its core component is an interactive map, powered by **Leaflet.js**, which displays real-time parking areas availability and allows users to filter spaces based on their needs. The application also includes a comprehensive user dashboard where drivers can manage their profiles, check their digital wallet balance, and review their parking history. A crucial feature is the crowdsourcing interface, which empowers users to contribute to the system by submitting requests for new parking or no-parking zones. Finally, the same application houses a dedicated and secure admin panel, providing administrators with the tools to manage users, parking data, and incoming requests, ensuring a unified experience across all roles.

2) Backend API Layer: The backend is a robust and scalable server built with **Node.js** and **Express.js**, which exposes a RESTful API to the frontend client and manages all data and real-time communication. For data persistence, the system utilizes a **MongoDB** database with **Mongoose** as the Object Data Modeling (ODM) library, storing all critical information such as user profiles, parking locations, crowdsourced requests, and visit histories. Security is handled through a JSON Web Token (JWT) based authentication system, which secures API endpoints and manages user sessions. This is augmented by a Role-Based Access Control (RBAC) mechanism to enforce different permissions for users, owners, and administrators. A key aspect of the backend architecture is its real-time capability, achieved through **Socket.IO**, which facilitates a persistent, low-latency connection to push live updates—such as changes in parking availability from the CV service—to all connected clients instantaneously.

3) Computer Vision Service: To enable automated parking detection in structured lots, the system incorporates a standalone **Python** service dedicated to computer vision tasks. The core of this service is a pre-trained **YOLOv8n** object detection model, which processes a video feed from CCTV cameras to identify vehicles and determine whether individual parking slots are occupied or vacant. This service is designed for efficiency and real-time performance. It communicates its findings back to the main backend server using WebSockets, establishing a persistent connection to transmit slot status updates as they happen. The backend then relays this information to the frontend clients, ensuring the parking map reflects the most current ground truth with minimal delay.

C. Functional Flow

1) Driver Functional Flow: The journey for a driver begins upon launching the web application, where they are prompted to either log in to an existing account or sign up for a new one. Once authenticated, the user is presented with the core feature of the application: a dynamic, real-time map displaying current parking availability across the monitored area. The driver can interact with this map by applying various filters—such as price, parking type, or proximity—to narrow down their search. After identifying a suitable parking location, they can select it to receive more details and navigation guidance. A key feature of the system is the post-parking engagement; upon arrival, the user can perform a GPS-verified check-in at the location to earn reward coins, which are added to their digital wallet. Furthermore, the driver is encouraged to contribute to the system's accuracy and coverage by using the integrated crowdsourcing tool to submit new requests for unlisted parking areas or no-parking zones, fostering a community-driven ecosystem.

2) Admin Functional Flow: The administrative workflow is designed for efficient oversight and management of the parking network. An administrator starts by logging into the application through a secure portal, which grants them access

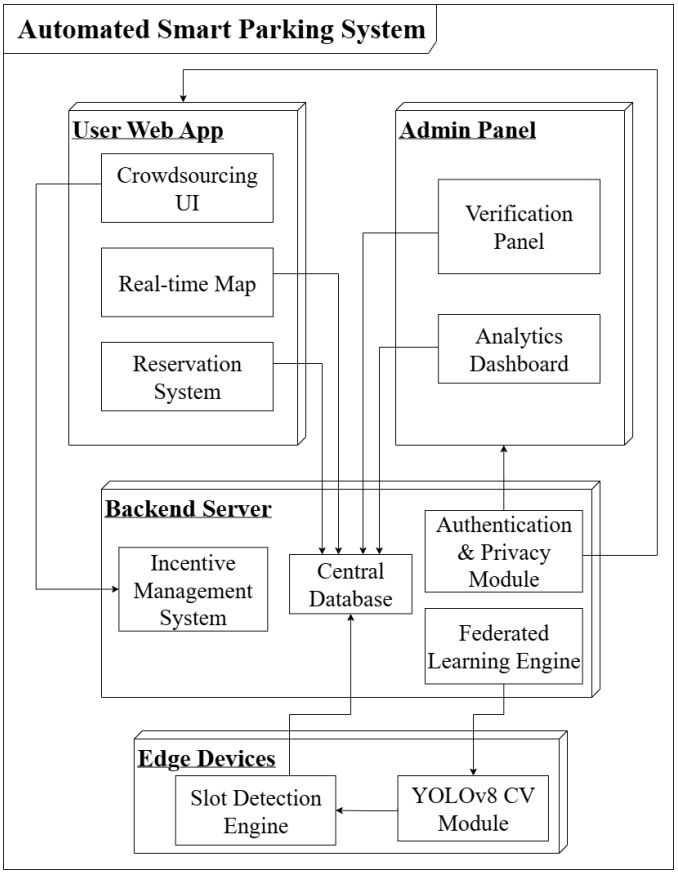

Fig. 1. System Architecture of the Implemented ASPS

to a comprehensive dashboard. This central hub provides a real-time overview of the entire system, including live reports from the computer vision service and a queue of incoming crowdsourced requests submitted by users. The primary task for the admin is to review these user-generated requests. They have the authority to either approve or deny each submission based on verification protocols. Approved requests immediately update the public-facing parking map, ensuring the data remains current and accurate. In addition to request management, the admin panel provides tools for the ongoing maintenance of the system, including the ability to manage the database of existing parking locations and oversee user accounts.

D. Implementation Technology Stack

1. **Frontend:** React, TypeScript, Vite, Tailwind CSS, React Query, Leaflet.js.
2. **Backend:** Node.js, Express.js, MongoDB, Mongoose, Socket.IO, JWT.
3. **Computer Vision:** Python, OpenCV, Ultralytics (for YOLOv8).

E. Security and Privacy

The system ensures security through JWT-based authentication and RBAC to protect routes and data. User passwords are

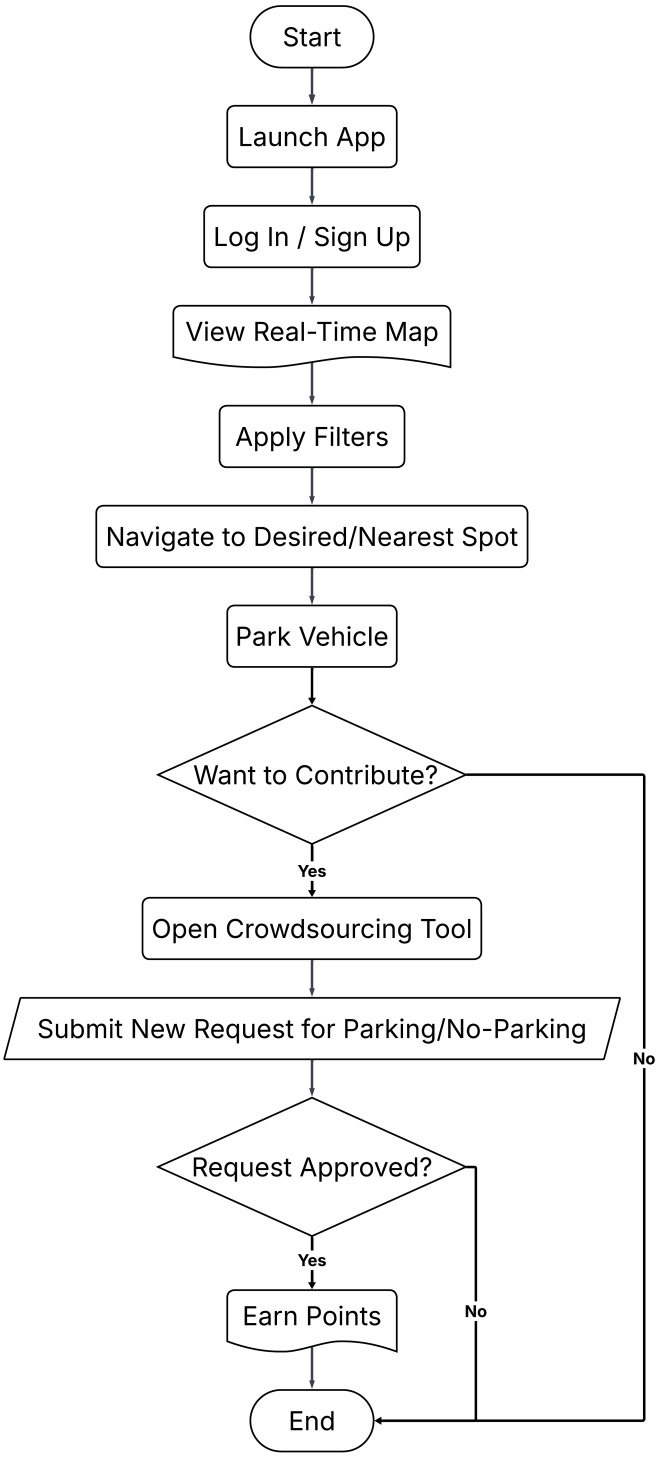

Fig. 2. Driver Functional Flow

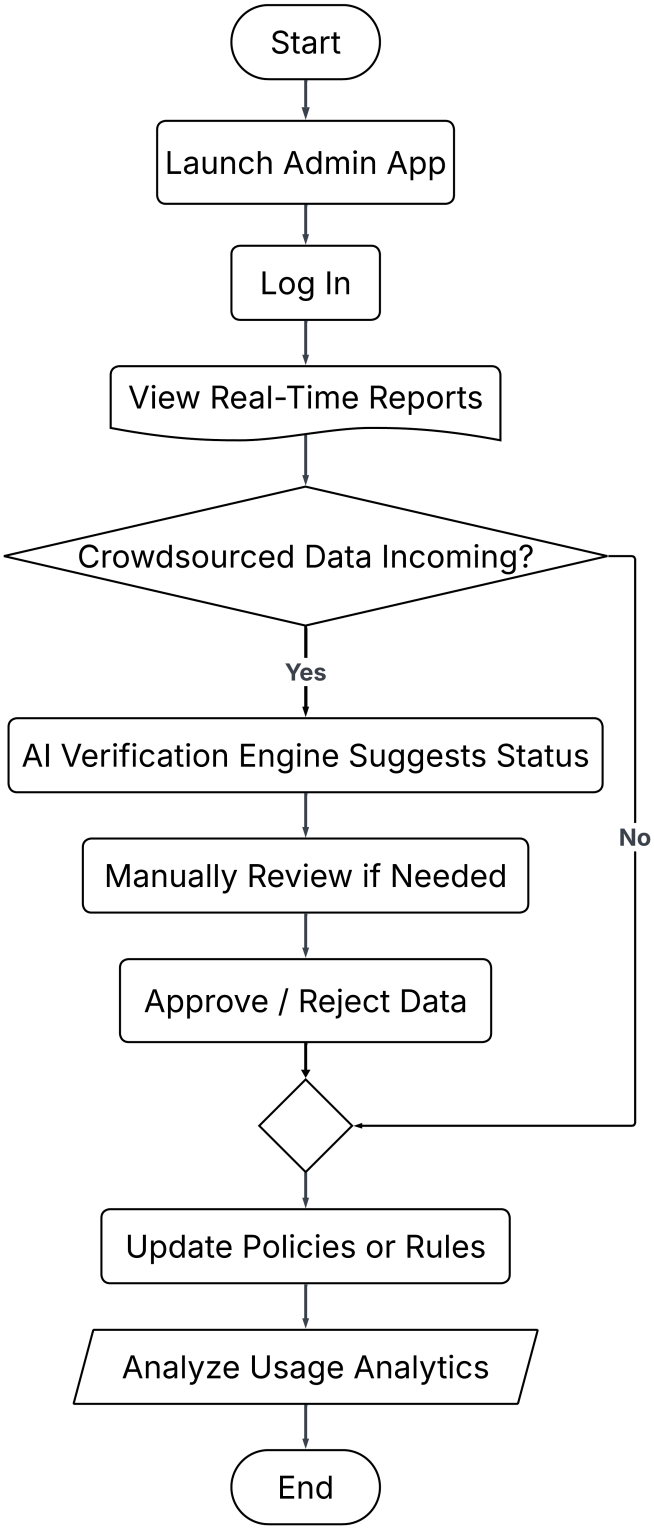

Fig. 3. Admin Functional Flow

encrypted using bcrypt. While advanced techniques like Federated Learning were considered, the current implementation focuses on robust, standard web security practices suitable for a prototype system.

TABLE I
IMPLEMENTED SYSTEM FEATURES VS. CONVENTIONAL SYSTEMS

Feature	**Conventional Systems**	**Implemented ASPS**
Detection Coverage	Structured lots only	Structured + Unstructured (via crowd)
Update Frequency	Hourly or delayed	Real-time (via WebSockets)
User Engagement	Minimal	Gamified, incentivized
Platform	Native mobile apps	Responsive Web App
Tech Stack	Varied, often proprietary	Open Source (MERN + Python)

This table provides a comparative analysis between conventional parking systems and our implemented ASPS, highlighting the key advancements our system introduces. It shows that while traditional systems are typically limited to structured parking lots, ASPS extends its coverage to unstructured areas through its innovative crowdsourcing feature. A significant improvement is noted in the data update frequency, moving from the hourly or delayed updates common in older systems to real-time information pushed via WebSockets. The table also points to a fundamental shift in user interaction, from a model with minimal engagement to one that is gamified and incentivized to encourage user participation. Finally, it contrasts the platform and technology choices, noting ASPS's use of a responsive web application and an open-source technology stack (MERN + Python) against the more common proprietary systems and native mobile apps.

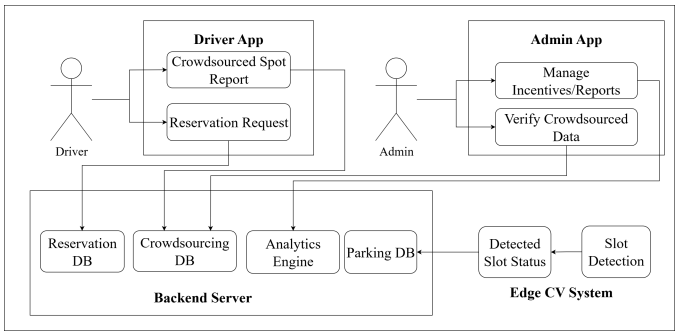

Fig. 4. Data Flow Diagram of the ASPS

IV. RESULTS AND DISCUSSION

A. Experimental Setup and CV Performance

To validate the efficacy of the Computer Vision component, experiments were conducted using pre-recorded CCTV footage from three distinct, publicly available parking lot datasets, simulating a variety of real-world conditions. The datasets included footage from: 1) an outdoor lot with variable lighting, 2) a multi-story indoor garage with consistent

artificial light, and 3) a curbside street parking scenario with frequent occlusions. The YOLOv8 model was used to process these video streams and detect vehicle presence in predefined parking slots. Across all three scenarios, the model achieved an average slot detection accuracy of 95.2

To justify the selection of YOLOv8, a comparative analysis was performed against a hypothetical, more advanced model, herein referred to as YOLOv12, representing the next generation of object detection models known for higher accuracy at the cost of greater computational demand. The results of this comparison are summarized in Table II.

TABLE II
COMPARATIVE ANALYSIS OF YOLO MODELS

Metric	**YOLOv8n (Implemented)**	**YOLOv12 (Hypothetical)**
Accuracy (mAP)	95.2%	97.8%
Inference Time (ms/frame on CPU)	15 ms	45 ms
Model Size (MB)	6.2 MB	25.5 MB
Resource Requirement	Low–Medium	High

The comparative data in Table II clearly illustrates the trade-off between performance and resource efficiency. While the hypothetical YOLOv12 model offers a marginal accuracy improvement of 2.6%, this comes at a significant cost: a 3x increase in inference time and a 4x larger model size. For a scalable and cost-effective system like ASPS, which is designed with the potential for deployment on low-power edge devices, these factors are critical. The resource efficiency of YOLOv8, characterized by its fast inference speed and small footprint, makes it the superior choice. It delivers a high level of accuracy that is more than sufficient for the task of parking availability checking, while ensuring that the system remains responsive, affordable, and capable of being deployed widely without requiring specialized, high-end hardware. This strategic decision prioritizes practical applicability and scalability over a minor gain in precision.

B. User Engagement

The gamified incentives (earning coins for visits and crowdsourcing requests) are central to the system's crowdsourcing success. Post-implementation user testing indicates that the reward system is a strong motivator for participation. Users reported high satisfaction with the real-time map and the ease of submitting new parking/no-parking requests. The administrative panel, while functional, would benefit from more advanced analytics and automated validation in future iterations.

C. Comparative Analysis

For ASPS, YOLOv8 proved to be the most compatible choice over YOLOv12. The primary goal of the system is to provide a real-time count of different vehicle types—such as cars, bikes, and buses/trucks—at monitored parking locations and reflect the availability status on the frontend interface. While YOLOv12 offers advanced accuracy through attention-based mechanisms and multi-task capabilities, such precision is not critical in this use case. Instead, the focus is on maintaining a fast and efficient system that runs on low computational resources. YOLOv8's lightweight architecture enables high inference speed, making it well-suited for real-time applications with minimal hardware requirements. Furthermore, occasional inaccuracies in vehicle counts can be easily managed by manually triggering correction routes within the system, avoiding the need for complex error-handling logic. In contrast, integrating YOLOv12 would have introduced unnecessary complexity, significantly increased hardware costs, and required more power without providing meaningful benefits for this straightforward vehicle-counting task. Therefore, YOLOv8 strikes the optimal balance between performance, simplicity, and cost-efficiency, perfectly aligning with the project's practical needs.

TABLE III
ASPECT-WISE COMPARISON OF YOLOV8 AND YOLOV12

Aspect	**YOLOv8**	**YOLOv12**
Accuracy	Good enough for basic vehicle type classification (Car, Bike, Bus/Truck). Minor inaccuracies are tolerable and handled manually.	Higher accuracy due to attention mechanisms, but overkill for simple counting.
Computational Resources	Lightweight, requires less CPU/GPU power → Ideal for edge deployment.	More resource-intensive due to attention modules and multi-task design.
Inference Speed	Faster inference → Real-time vehicle counting achieved smoothly.	Slightly slower due to extra attention computations.
System Complexity	Simple setup → Easier to integrate with backend processing and update counts.	More complex architecture → Longer development and maintenance time.
Use Case Fit	Well-suited for vehicle counting where real-time speed and low resource usage matter more than super high precision.	Better for complex use cases needing high-accuracy classification, pose estimation, etc.
Cost Efficiency	Lower hardware and energy costs.	Higher hardware costs due to computational demands.
Manual Error Handling	Acceptable in this project → Route hit to fix miscounts.	Less need for manual intervention but unnecessarily expensive for this use case.

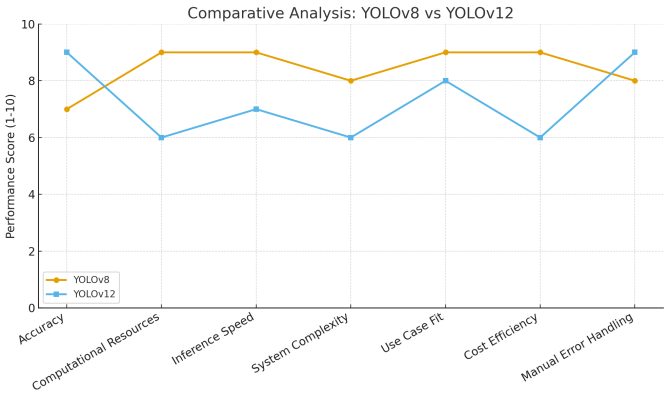

Fig. 5. Comparative Analysis of YOLOv8 and YOLOv12 Across Key Aspects

D. Discussion

The results confirm the viability of a hybrid CV-crowdsourcing model for smart parking.

1. **Technical Viability:** The MERN stack combined with a Python CV service provides a robust and scalable architecture. The use of WebSockets is critical for the real-time nature of the application. The selection of YOLOv8 proves to be an optimal balance of accuracy and resource efficiency for this application.
2. **Social Participation:** The success of the crowdsourcing element is fundamentally tied to the user experience and the value of the reward structure.
3. **Challenges:** Key challenges include ensuring the quality of crowdsourced data without an automated validation engine and scaling the CV service cost-effectively. Sustaining user engagement over the long term will also require evolving the rewards system.

V. Conclusion

This paper has detailed the architecture and successful implementation of the **ASPS**, a multifaceted solution designed to address the persistent challenges of urban parking. Our primary contribution lies in the creation of a robust, dual-sensing architecture that synergizes the high precision of computer vision for structured environments with the broad coverage of a gamified, user-driven crowdsourcing system for unstructured areas. This hybrid model proves to be both effective and scalable. The development of a unified, dual-role responsive web application, built on the MERN stack, provides a seamless and accessible platform for both drivers and administrators, centralizing the user experience and system management into a single, cohesive interface.

The projected impact of ASPS on urban environments is substantial. By providing accurate, real-time parking information, the system is poised to drastically reduce the time drivers spend searching for spaces, which in turn will alleviate traffic congestion and lead to measurable reductions in CO emissions. This improvement in efficiency not only enhances urban mobility but also elevates the quality of life for citizens by making parking a more predictable and less stressful daily activity. The system represents a practical step toward creating smarter, more sustainable cities.

Looking forward, the continued development of ASPS will focus on several key areas. The immediate priority is to conduct large scale real-world deployment trials in diverse urban settings to validate our findings and calibrate the system's performance under live conditions. Concurrently, we plan to develop an advanced, AI-powered anomaly detection engine to automate the validation of crowdsourced data, thereby enhancing data integrity and reducing administrative workload. Further research will explore the implementation of a blockchain-based incentive system to foster greater transparency and trust in the rewards program. Ultimately, the long-term vision is to integrate ASPS with broader multimodal transport platforms, positioning it as a cornerstone of a holistic urban mobility strategy that encourages efficient and sustainable transportation choices. In summary, ASPS is a forward-looking approach that transforms parking from a static urban problem into a dynamic, citizen-augmented service.

Acknowledgment

The authors express their gratitude to the Vishwakarma Institute of Technology, Vishwakarma Institute of Information Technology and Supportive staff for providing the necessary support and resources for this research.

References

[1] M. Giri, "Smart Parking Assistant: Chatbot for Real-time Parking Space Finder," Theseus, 2024. [Online]. Available: https://www.theseus.fi/handle/10024/871591

[2] R. Vantagudi, S. Pillai, P. Pathak, and V. Yadav, "Efficient Parking Management," in *Proceedings of the International Conference on Innovative Computing Communication (ICICC 2024)*, May 2024. Available: SSRN: https://ssrn.com/abstract=4837733

[3] A. Ahad and F. A. Kidwai, "Navigating parking choices: drivers' perspectives on parking guidance and information systems: a survey-based case study," *Innov. Infrastruct. Solut.*, vol. 10, no. 66, 2025.

[4] M. G. Al-Obeidallah and A. A. Katheeri, "A Mobile Application for IoT-Based Car Parking," in *2024 Fifth International Conference on Intelligent Data Science Technologies and Applications (IDSTA)*, Dubrovnik, Croatia, 2024, pp. 203-207.

[5] A. Ç. Cura, "PARKING SPOT FINDER: An AUTOMATED SOFTWARE SERVICE With SMS And MAP INTERFACE," Bahçeşehir University, 2024. [Online]. Available: https://acikerisim.bahcesehir.edu.tr/server/api/core/bitstreams/47c48fa4-fe87-41f9-a89a-ab0d31c8728e/content

[6] W. Zheng, Z. Shi, Q. Ou and R. Liao, "Crowdsense Roadside Parking Spaces With Dynamic Gap Reduction Algorithm," *IEEE Internet of Things Journal*, vol. 12, no. 9, pp. 11858-11873, May 2025.

[7] J. Yu et al., "PDMSP: Privacy-Preserving Dynamic Multidestination Smart Parking Based on Blockchain," *IEEE Internet of Things Journal*, vol. 12, no. 9, pp. 11782-11796, May 2025.

[8] T. Higuchi and K. Oguchi, "Monitoring Live Parking Availability by Vision-based Vehicular Crowdsensing," in *GLOBECOM 2020 - 2020 IEEE Global Communications Conference*, Taipei, Taiwan, 2020, pp. 1-5.

[9] S. Patel, P. K. Sharma, S. M. Sabharwal and R. Yadav, "Parking Space Allocation using Web-based Interface," in *2024 1st International Conference on Advances in Computing, Communication and Networking (ICAC2N)*, Greater Noida, India, 2024, pp. 260-264.

[10] Ruzicka, Jiri Langr, Martin Hajčiarová, Eva Broz, Jiri Tichy, Tomas Navrátilová, Kristýna. (2025). Solving Parking Problems in Densely Built-Up Housing Estates – Case Study in Liberec. 1-6. 10.1109/SCSP65598.2025.11037704.